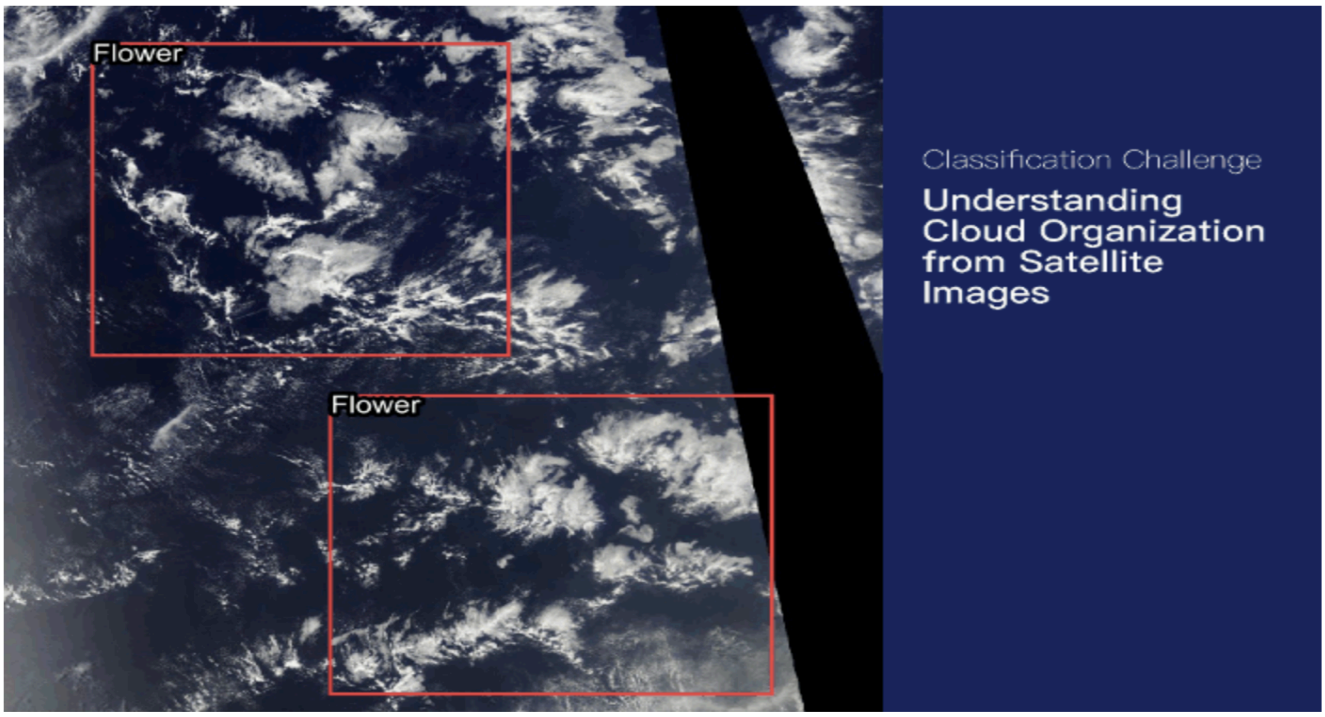
DataScientest • com



Segmentation of cloud regions

Author: Fatemeh Tayebi Naeempour
Promotion: Bootcamp June 2026
Supervision: Yohan Cohen (DataScientest)
Project Presentation: 22 September 2026

Project Introduction / Background

Clouds play a crucial role in the Earth's climate system by influencing energy balance and weather patterns. However, their complex structures and organization make them difficult to represent accurately in climate models. Understanding different cloud patterns can help scientists improve climate predictions and reduce uncertainties in future climate projections.

In this project, we aim to use satellite images and machine learning techniques to segment and identify different cloud organization patterns. By accurately detecting cloud types and their spatial distribution, this work can contribute to a better understanding of cloud organization and atmospheric processes.

Chapter 1:

EDA – Exploratory Data Analysis

1.1 Dataset Overview

The dataset used in this project is the Kaggle *Understanding Clouds from Satellite Images* dataset. It consists of satellite images and corresponding cloud segmentation annotations for four cloud categories: **Fish, Flower, Gravel, and Sugar**.

The dataset contains **5,546 unique satellite images** and **22,184 annotation records**, with **5,546 records for each cloud category**. The segmentation annotations are provided in **Run-Length Encoding (RLE)** format through the `EncodedPixels` column of the training CSV file.

The original dataset images have a resolution of **1400 × 2100 pixels**. For this project, the images were processed and resized to **576 × 384 pixels**, corresponding to a NumPy shape of **384 × 576 × 3**.

The dataset is provided through several files:

- `train.csv`: contains the RLE-encoded segmentation masks for each image and cloud-label pair in the training set.
- `train_images.zip`: contains the training satellite images used for model development.
- `test_images.zip`: contains the test satellite images.
- `sample_submission.csv`: provides an example of the required output format for segmentation predictions. This file was not used in the current project.

Not all annotation records contain a segmentation mask. Approximately **53.35% of the annotation records contain valid RLE masks**, while the remaining records have missing `EncodedPixels` values. These missing values indicate that no segmentation mask is provided for that particular cloud category in the corresponding image.

The satellite images were obtained from **NASA Worldview** and were captured by the **TERRA and AQUA** polar-orbiting satellites using the **MODIS** instrument. Because an image may be constructed from two satellite orbits, an uncovered area between successive orbits can appear as a **black band**. This area is not part of the valid image region and must not be considered as a cloud prediction area.

The ground-truth annotations were created through a crowdsourcing process involving **68 scientists**. Each image was labeled by approximately three scientists, and the final ground

truth was obtained from the union of their annotations after removing the black band areas.

The dataset therefore provides satellite imagery and pixel-level cloud annotations that can be used to formulate the project as a **multi-label image segmentation task**.

1.2 Dataset Structure

The training annotations are stored in a CSV file containing two main columns: `Image_Label` and `EncodedPixels`.

Table 1.1 Initial dataframe of the training dataset

	Image_Label	EncodedPixels
0	0011165.jpg_Fish	20056 257 20440 257 20824 257 21208 257 21592 ...
1	0011165.jpg_Flower	102135 1 102245 275 102629 275 103013 275 1033...
2	0011165.jpg_Gravel	NaN
3	0011165.jpg_Sugar	NaN
4	002be4f.jpg_Fish	17668 241 18051 242 18435 242 18819 242 19203 ...

The `Image_Label` column contains both the image identifier and the corresponding cloud category. To make the data easier to process, this column was split into two separate columns: `Image_id` and `Cloud_Type`.

Table 1.2 Dataset structure after separating image identifiers and cloud categories

	Image_id	Cloud_Type	EncodedPixels
0	0011165.jpg	Fish	20056 257 20440 257 20824 257 21208 257 21592 ...
1	0011165.jpg	Flower	102135 1 102245 275 102629 275 103013 275 1033...
2	0011165.jpg	Gravel	NaN
3	0011165.jpg	Sugar	NaN
4	002be4f.jpg	Fish	17668 241 18051 242 18435 242 18819 242 19203 ...

For example, an entry such as `0011165.jpg_Fish` is separated into the image identifier `0011165.jpg` and the cloud category `Fish`.

Each image can appear up to four times in the annotation file, once for each cloud category: Fish, Flower, Gravel, and Sugar.

The `EncodedPixels` column contains the segmentation mask information using Run-Length Encoding (RLE). When a cloud of a given category is present and annotated, its pixel locations are stored as an RLE sequence. Missing values in this column indicate that no mask is available for that particular image-category pair.

The RLE-encoded annotations were later decoded and converted into multi-channel segmentation masks for model training. The detailed preprocessing steps are described in Chapter 2.

This structure allows each satellite image to be associated with a corresponding pixel-level target containing the segmentation information for all four cloud categories.

1.3 Data Exploration and Visualization

1.3.1 Class Distribution

The distribution of available segmentation masks was analyzed for the four cloud categories: Fish, Flower, Gravel, and Sugar. The analysis shows that the number of annotated masks is not exactly equal across the four classes.

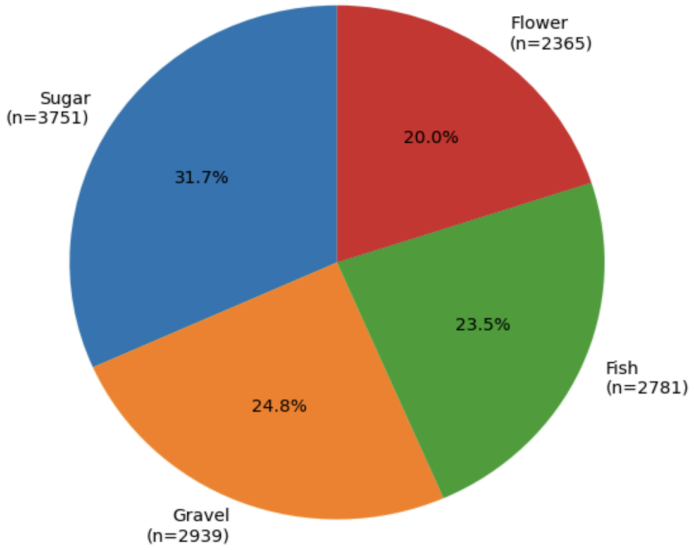


Figure 1.1: Distribution of available segmentation masks by cloud type

The pie chart shows the proportion of available masks belonging to each cloud category. Sugar represents the largest proportion, while Flower has the smallest proportion.

Overall, the dataset shows a moderate class imbalance, with Sugar having the highest number of available masks and Flower the lowest. Because the imbalance is not severe, no additional class-balancing technique was applied during preprocessing or model training. In cases of stronger class imbalance, techniques such as class-weighted loss could be used to give greater importance to underrepresented classes. However, for this dataset, the observed imbalance was considered moderate enough to allow the model to learn from the original class distribution without introducing additional weighting.

1.3.2 Mask Availability and Cloud Categories per Image

The training dataset contains 22,184 image-category annotation records. Among these records, 11,836 contain segmentation masks, while 10,348 have missing EncodedPixels values. Therefore, 53.35% of the annotation records contain an available segmentation mask, while 46.65% do not. A missing value indicates that no annotated segmentation mask is available for that particular cloud category in the corresponding image. However, this does not mean that the image contains no clouds, since each image can contain one or more of the four cloud categories. The number of cloud categories represented by available annotations was also analyzed for each image.

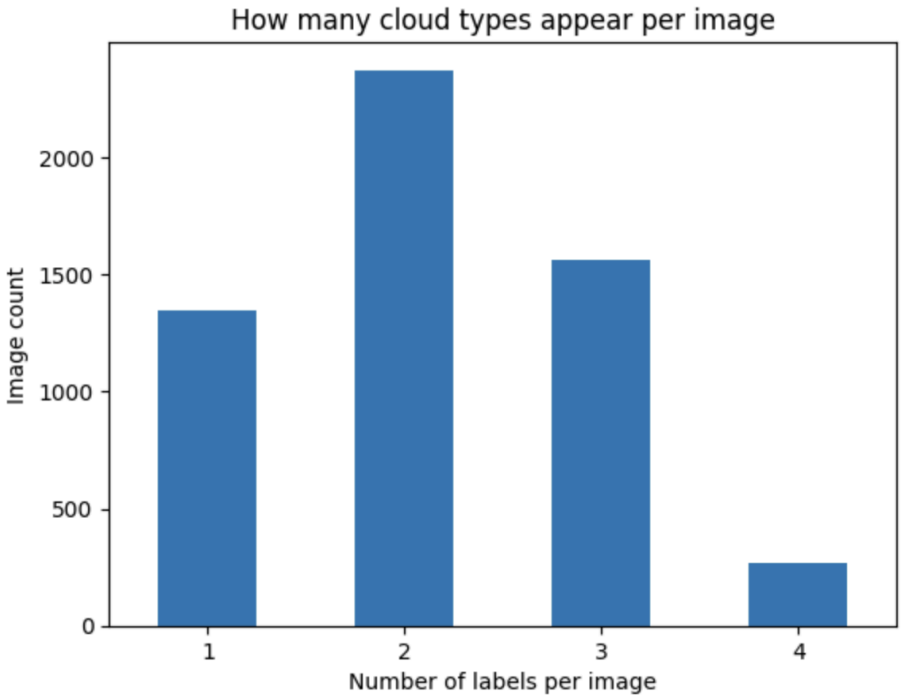


Figure 1.2: Number of Cloud Types per Image

The count plot shows the number of cloud types represented by available annotations in each image. The results show that images contain between one and four cloud types. Most images contain two cloud types, followed by three and one cloud type, while only a small number of images contain all four cloud types.

This distribution shows that most satellite images contain multiple cloud types, highlighting the complexity of the segmentation task. The model therefore needs to identify and distinguish several cloud categories within the same image.

1.3.3 Sensor Gap Analysis

During the exploration of the satellite images, a black band was observed in some images. This band is caused by an uncovered area between successive satellite orbits and represents a sensor gap rather than a valid image region.

The sensor gap was analyzed to understand the structure of the satellite images and to distinguish the black band from actual cloud regions.

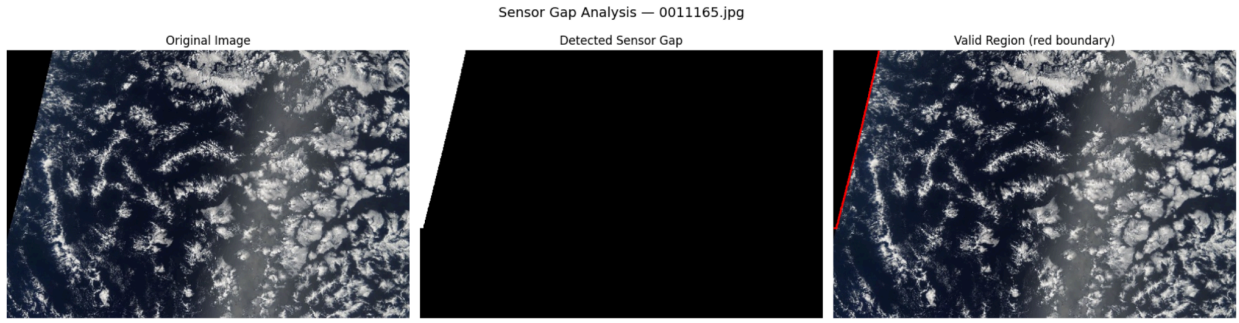


Figure 1.3: Example of sensor gap detection

Left: Original satellite image.
Middle: Detected sensor gap (white = gap).
Right: Valid image region with its boundary.

The visualization shows that the black band does not contain valid satellite information and should not be interpreted as a cloud region. The ground truth annotations already exclude these areas.

In the modeling stage, no additional sensor-gap mask was applied during training or prediction. This approach avoids introducing errors from heuristic pixel-based gap detection and keeps the modeling process focused on the provided ground truth annotations.

Chapter 2

Data Pre-processing

2.1 RLE Decoding

The segmentation masks in the training dataset are provided in Run-Length Encoding (RLE) format through the EncodedPixels column. RLE represents the locations of segmented pixels using pairs of **start positions** and **run lengths**.

To use these annotations for image segmentation, the RLE-encoded values were decoded into binary masks. Each decoded mask has the same spatial dimensions as the processed images, 384 × 576 pixels. Pixels belonging to the corresponding cloud category are assigned a value of 1, while the remaining pixels are assigned a value of 0.

Missing EncodedPixels values were treated as empty masks, since they indicate that no segmentation mask is available for that image-category pair.

The decoded binary masks were then used as the basis for creating the multi-channel ground truth masks described in the next section.

2.2 Multi-channel Mask Creation

After decoding the RLE annotations, the binary masks were combined into a single multi-channel mask for each satellite image. Four channels were created, one for each cloud category: Fish, Flower, Gravel, and Sugar.

Each channel represents the segmentation mask of one cloud category, with 1 indicating the presence of the corresponding cloud type and 0 indicating background pixels. Therefore, each processed image is associated with a four-channel ground truth mask with a shape of 384 × 576 × 4.

This representation allows the model to predict the four cloud categories independently for each pixel and provides the ground truth required for multi-label cloud segmentation.

2.3 Image and Ground Truth Mask Visualization

To verify the segmentation annotations, a selected satellite image was visualized together with its corresponding ground truth masks. The masks were displayed separately for each cloud category to examine their spatial alignment with the original image.

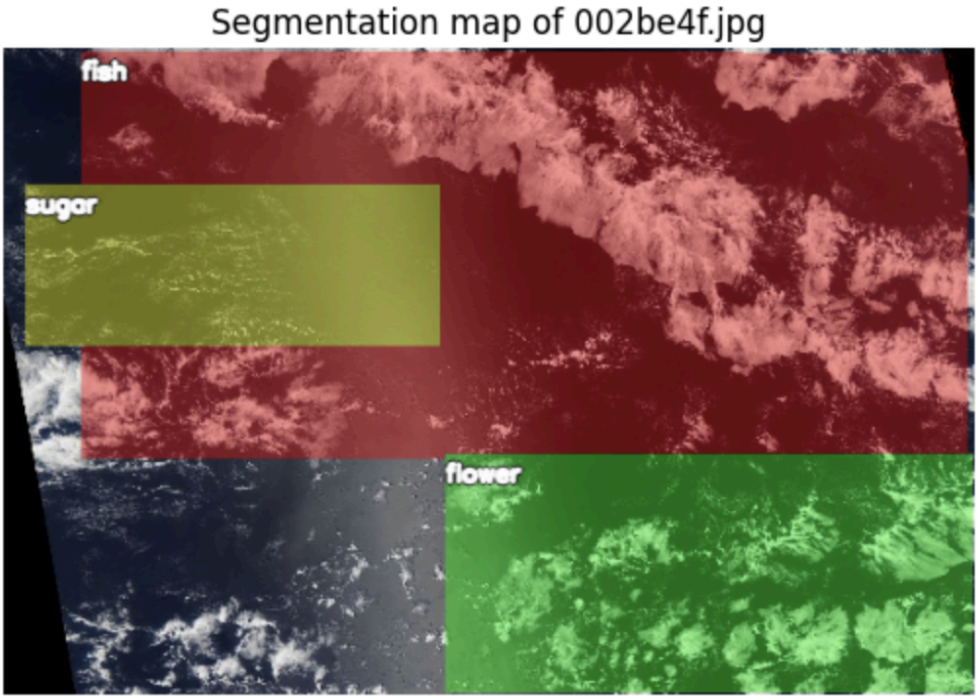


Figure 2.1: Satellite image 002be4f.jpg with ground truth segmentation masks

The visualization shows the satellite image 002be4f.jpg together with its available ground truth segmentation masks. The masks illustrate the spatial distribution of the annotated cloud regions and their alignment with the original image.

The black band visible in the lower-left corner represents the sensor gap. It is not part of the valid image region, and the ground truth annotations already exclude this area.

This visualization provides a visual check of the segmentation annotations and confirms their alignment with the satellite image before model training.

2.4 Image Resizing

The original satellite images have a resolution of 1400 × 2100 pixels. To reduce the computational cost and make the images suitable for model training, the images were resized to 576 × 384 pixels.

The same spatial dimensions were used for the corresponding ground truth masks to ensure that the images and masks remained correctly aligned. After resizing, each image had a shape of 384 × 576 × 3, while each ground truth mask had a shape of 384 × 576 × 4.

This resizing step reduced the input size while preserving the spatial information required for cloud segmentation and made the dataset more suitable for efficient model training.

2.5 Normalization

After resizing, the satellite images were normalized before being used as input to the neural networks. ImageNet normalization was applied using the mean values (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225) for the RGB channels.

This normalization was used because the models use ImageNet-pretrained encoders. It provides input values with a distribution compatible with the pretrained weights and supports more stable model training.

The ground truth masks were not normalized. They were kept as binary values, where 1 represents the presence of a cloud category and 0 represents its absence.

2.6 Data Augmentation

Data augmentation was applied only to the training data to increase the diversity of the satellite images and improve the model's ability to generalize to unseen images. The validation data was not augmented and was only resized and normalized.

The training augmentation included horizontal and vertical flipping, shifting, scaling, and rotation. Additional transformations included changes in brightness and contrast, gamma adjustment, CLAHE, Gaussian noise, Gaussian blur, and motion blur.

The same spatial transformations were applied to both the images and their corresponding masks to preserve their alignment. Data augmentation was used only during training and was not applied to validation or test images.

2.7 Train/Validation Split

The dataset was divided into two subsets: **90% for training and 10% for validation**. The split was performed at the image level using a fixed random state of 42.

- **Training images:** 4,991
- **Validation images:** 555
- **Total:** 5,546 images

The validation set was used to evaluate the model during training and to monitor its performance on unseen images.

2.8 Data Loaders

The training and validation datasets were loaded using PyTorch `DataLoader`. A batch size of **8** was used for both datasets.

The training DataLoader shuffled the training samples to provide a different order of images during each epoch. The validation DataLoader did not shuffle the data because the order of validation samples does not need to be randomized.

The DataLoaders were configured with `num_workers=0` and `pin_memory=True` to control data loading and support efficient data transfer to the GPU.

2.9 Batch Size

A batch size of **8** was used for both the training and validation datasets. This means that the model processes **8 images at a time** before updating its weights during training.

Using a batch size of 8 provides a balance between computational efficiency and GPU memory usage.

2.10 Final Data Preparation

After preprocessing, the images have the shape (3, 384, 576) and the corresponding masks have the shape (4, 384, 576). The images contain three RGB channels, while the masks contain four channels corresponding to Fish, Flower, Gravel, and Sugar.

The data is then loaded by the DataLoader in batches of 8. Therefore, the final input shapes used by the model are:

- Images: [8, 3, 384, 576]
- Masks: [8, 4, 384, 576]

Here, 8 represents the batch size, 3 represents the RGB channels, and 4 represents the four cloud classes.

Chapter 3

Model Development

3.1 Model Architecture – U-Net with ResNet50

The first deep learning model used in this project was a U-Net architecture with a ResNet50 encoder. The model was implemented using the `segmentation_models_pytorch` library.

The ResNet50 encoder was initialized with ImageNet-pretrained weights. It receives RGB satellite images with three input channels and extracts hierarchical features from the images. The U-Net decoder then uses these features to recover the spatial information required for pixel-level segmentation.

The model produces four output channels corresponding to the four cloud categories: **Fish, Flower, Gravel, and Sugar**. The model uses `activation=None`, which means that it produces raw logits. The sigmoid function is applied later to convert these outputs into probabilities for loss calculation and prediction.

```
==========================================================================================
Layer (type:depth-idx)                             Output Shape              Param #
==========================================================================================
Unet                                               [1, 4, 384, 576]          --
├─ResNetEncoder: 1-1                               [1, 3, 384, 576]          --
│    └─Conv2d: 2-1                                 [1, 64, 192, 288]         9,408
│    └─BatchNorm2d: 2-2                            [1, 64, 192, 288]         128
│    └─ReLU: 2-3                                   [1, 64, 192, 288]         --
│    └─MaxPool2d: 2-4                              [1, 64, 96, 144]          --
│    └─Sequential: 2-5                             [1, 256, 96, 144]         --
│    │    └─Bottleneck: 3-1                        [1, 256, 96, 144]         75,008
│    │    └─Bottleneck: 3-2                        [1, 256, 96, 144]         70,400
│    │    └─Bottleneck: 3-3                        [1, 256, 96, 144]         70,400
│    └─Sequential: 2-6                             [1, 512, 48, 72]          --
│    │    └─Bottleneck: 3-4                        [1, 512, 48, 72]          379,392
│    │    └─Bottleneck: 3-5                        [1, 512, 48, 72]          280,064
│    │    └─Bottleneck: 3-6                        [1, 512, 48, 72]          280,064
│    │    └─Bottleneck: 3-7                        [1, 512, 48, 72]          280,064
│    └─Sequential: 2-7                             [1, 1024, 24, 36]         --
│    │    └─Bottleneck: 3-8                        [1, 1024, 24, 36]         1,512,448
│    │    └─Bottleneck: 3-9                        [1, 1024, 24, 36]         1,117,184
│    │    └─Bottleneck: 3-10                       [1, 1024, 24, 36]         1,117,184
│    │    └─Bottleneck: 3-11                       [1, 1024, 24, 36]         1,117,184
│    │    └─Bottleneck: 3-12                       [1, 1024, 24, 36]         1,117,184
│    │    └─Bottleneck: 3-13                       [1, 1024, 24, 36]         1,117,184
│    └─Sequential: 2-8                             [1, 2048, 12, 18]         --
│    │    └─Bottleneck: 3-14                       [1, 2048, 12, 18]         6,039,552
│    │    └─Bottleneck: 3-15                       [1, 2048, 12, 18]         4,462,592
│    │    └─Bottleneck: 3-16                       [1, 2048, 12, 18]         4,462,592
├─UnetDecoder: 1-2                                 [1, 16, 384, 576]         --
│    └─Identity: 2-9                               [1, 2048, 12, 18]         --
│    └─ModuleList: 2-10                            --                        --
│    │    └─UnetDecoderBlock: 3-17                 [1, 256, 24, 36]          7,668,736
│    │    └─UnetDecoderBlock: 3-18                 [1, 128, 48, 72]          1,032,704
│    │    └─UnetDecoderBlock: 3-19                 [1, 64, 96, 144]          258,304
│    │    └─UnetDecoderBlock: 3-20                 [1, 32, 192, 288]         46,208
│    │    └─UnetDecoderBlock: 3-21                 [1, 16, 384, 576]         6,976
├─SegmentationHead: 1-3                            [1, 4, 384, 576]          --
│    └─Conv2d: 2-11                                [1, 4, 384, 576]          580
│    └─Identity: 2-12                              [1, 4, 384, 576]          --
│    └─Activation: 2-13                            [1, 4, 384, 576]          --
│    │    └─Identity: 3-22                         [1, 4, 384, 576]          --
==========================================================================================
```

Table 3.1. Detailed architecture of the U-Net with ResNet50

The architecture contains **32,521,540 trainable parameters**. All model parameters are trainable, with **0 non-trainable parameters**.

Table 3.1 presents the detailed architecture of the U-Net with ResNet50, including the layers, output shapes, and number of parameters at each stage.

The architecture consists of three main parts: the **ResNet50 encoder**, the **U-Net decoder**, and the **segmentation head**.

The ResNet50 encoder progressively reduces the spatial resolution of the input image from `384 × 576` to `12 × 18`, while increasing the number of feature channels from 64 to 2048. This allows the model to learn increasingly complex and high-level features from the satellite images.

The U-Net decoder then gradually restores the spatial resolution from `12 × 18` back to `384 × 576`. The decoder blocks reduce the number of channels from 256 to 16 while recovering spatial information needed for accurate segmentation.

Finally, the segmentation head uses a convolutional layer to produce **4 output channels**, one for each cloud category. Therefore, the final output has the shape `[1, 4, 384, 576]`.

3.2 Loss Function

A custom **Dice-BCE loss function** was used to train the model. It combines **Binary Cross-Entropy (BCE)** loss and **Dice loss**.

The total loss was calculated as a weighted combination of the two components, with a higher weight assigned to Dice loss:

- BCE loss weight: **0.3**
- Dice loss weight: **0.7**

The Dice component helps the model improve the overlap between the predicted cloud regions and the ground-truth masks, while BCE helps classify each pixel correctly.

The final loss was calculated as:

Total Loss = 0.3 × BCE Loss + 0.7 × Dice Loss

3.3 Optimizer and Learning Rate

The **AdamW optimizer** was used to update the model parameters during training. The initial learning rate was set to $\mathbf{8 \times 10^{-5}}$, and the weight decay was set to $\mathbf{1 \times 10^{-4}}$.

AdamW was selected to provide stable parameter updates while weight decay helps reduce overfitting and improve generalization.

A **ReduceLROnPlateau** scheduler was also used. When the validation loss stopped improving, the learning rate was reduced by a factor of **0.5** after **3 consecutive epochs** without improvement.

3.4 Training Configuration

The model was trained for a maximum of **25 epochs** with a batch size of **8**. During training, the model was evaluated on the validation set after each epoch.
Early stopping was used with a patience of **5 epochs**. If the validation performance did not improve for five consecutive epochs, training was stopped to reduce unnecessary training and help prevent overfitting.
The best-performing model was saved during training based on its validation Dice score. The saved checkpoint was then used for the final validation evaluation and test predictions.

3.5 Training Results

The model was trained for **25 epochs**. During training, both training loss and validation loss were monitored, together with the validation Dice score.

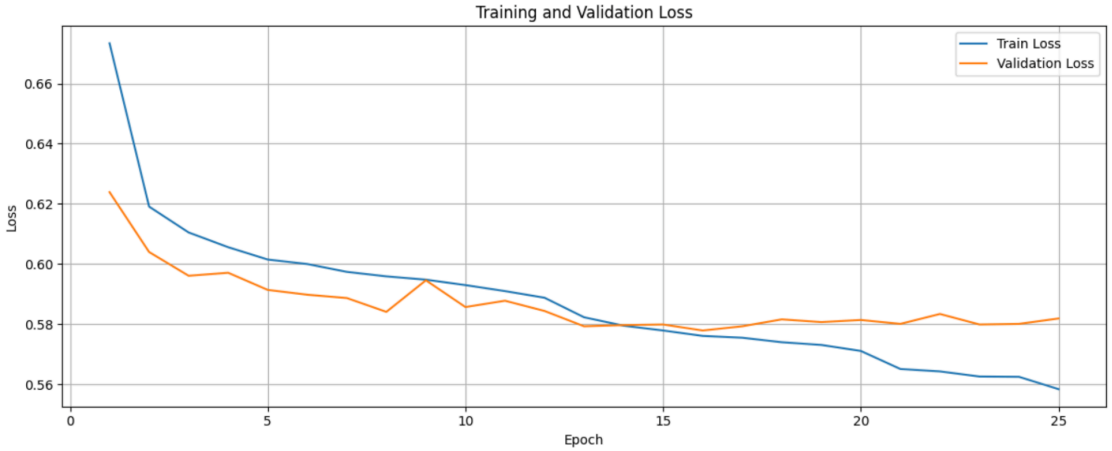


Figure 3.1. Training and Validation Loss across 25 Epochs

The training loss gradually decreased from **0.6734** in the first epoch to **0.5584** in the final epoch. The validation loss also improved during training, reaching its lowest value of approximately **0.5779** at epoch 16, before showing small fluctuations in the later epochs.

The validation Dice score generally improved during training and reached **0.5317** at epoch 25. The training results indicate that the model continued to learn throughout the training process, although the validation performance fluctuated between epochs. Figure 3.1 shows the training and validation loss curves across the 25 epochs.

3.6 Model Evaluation

After training, the best-performing U-Net with ResNet50 model was evaluated on the validation set. The evaluation was based on the **Dice score** and **Intersection over Union (IoU)**.

The overall **Macro Dice score was 0.6118**, while the **Mean IoU was 0.4432**. These results indicate that the model achieved a moderate level of segmentation performance on the validation images.

The Dice score was also calculated separately for each cloud category. **Flower** achieved the highest Dice score of **0.6962**, while **Fish** had the lowest score of **0.5478**. The results for Gravel and Sugar were **0.5729** and **0.6301**, respectively.

Cloud Class	Dice Score	IoU
Fish	0.5478	0.3772
Flower	0.6962	0.5340
Gravel	0.5729	0.4014
Sugar	0.6301	0.4600
Macro / Mean	**0.6118**	**0.4432**

The higher performance for Flower suggests that this class was easier for the model to segment, while Fish was more challenging. Overall, the model was able to detect the four cloud categories, but the segmentation quality varied between classes.

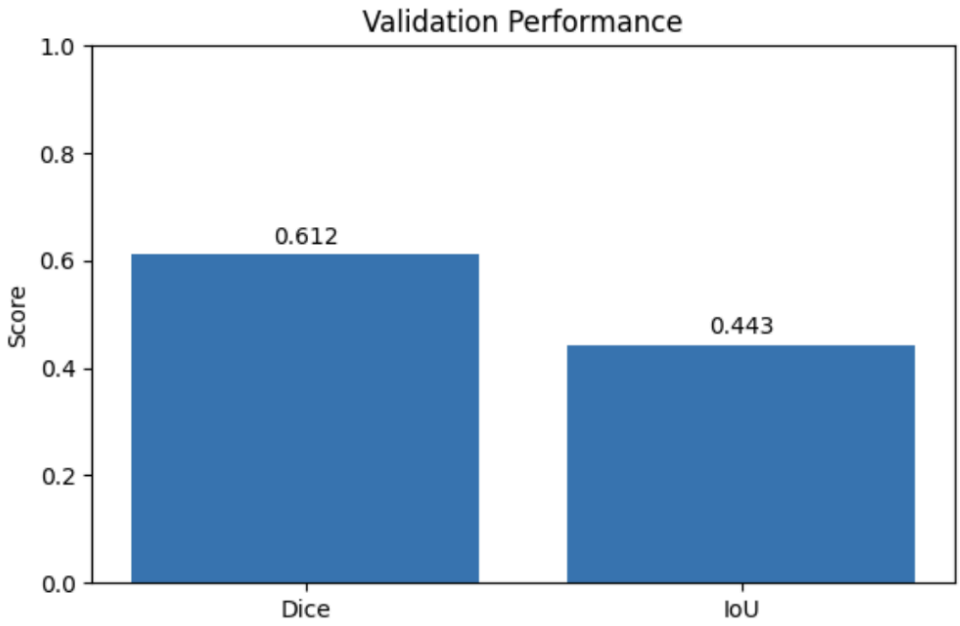


Figure 3.2. Validation Performance of the U-Net with ResNet50

Figure 3.2 compares the overall Dice score and Mean IoU on the validation set.

3.7 Prediction Visualization

To visually evaluate the segmentation results, predictions from the trained model were compared with the original satellite images and the corresponding ground-truth masks.

Four validation images were selected for visualization. For each image, the original satellite image, the ground-truth mask contours, and the predicted mask contours are shown side by side.

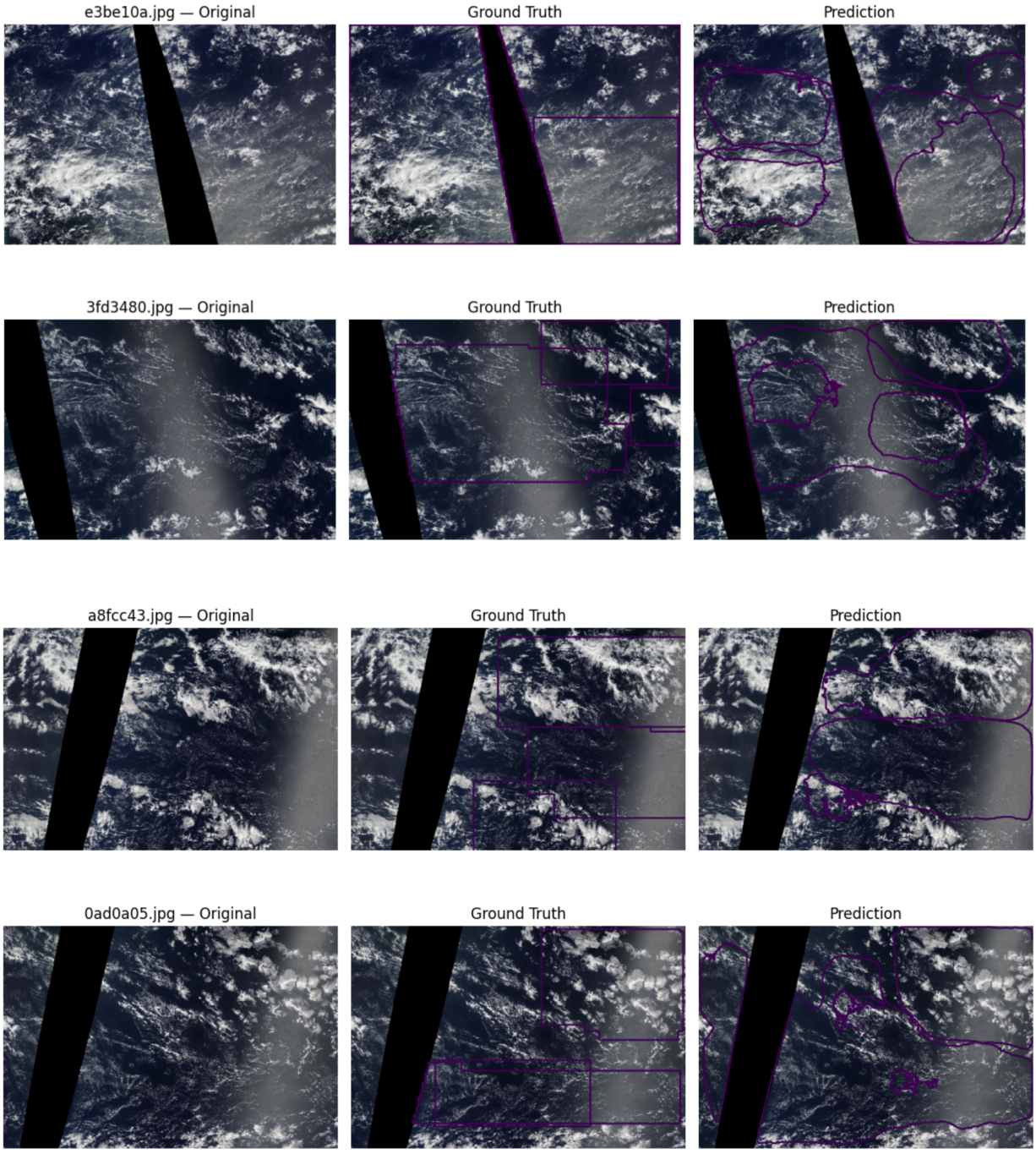


Figure 3.3. Comparison of Satellite Images, Ground Truth Masks, and Model Predictions

The visual comparison shows that the model was able to detect the main cloud regions in the selected validation images. However, some differences can be observed between the predicted contours and the ground-truth contours, particularly in the boundaries and coverage of some cloud regions. Some predicted regions are smaller or do not completely overlap with the corresponding ground-truth regions.

3.8 Test Set Prediction

After evaluating the model on the validation set, the selected U-Net with ResNet50 model was used to generate predictions for the test images.

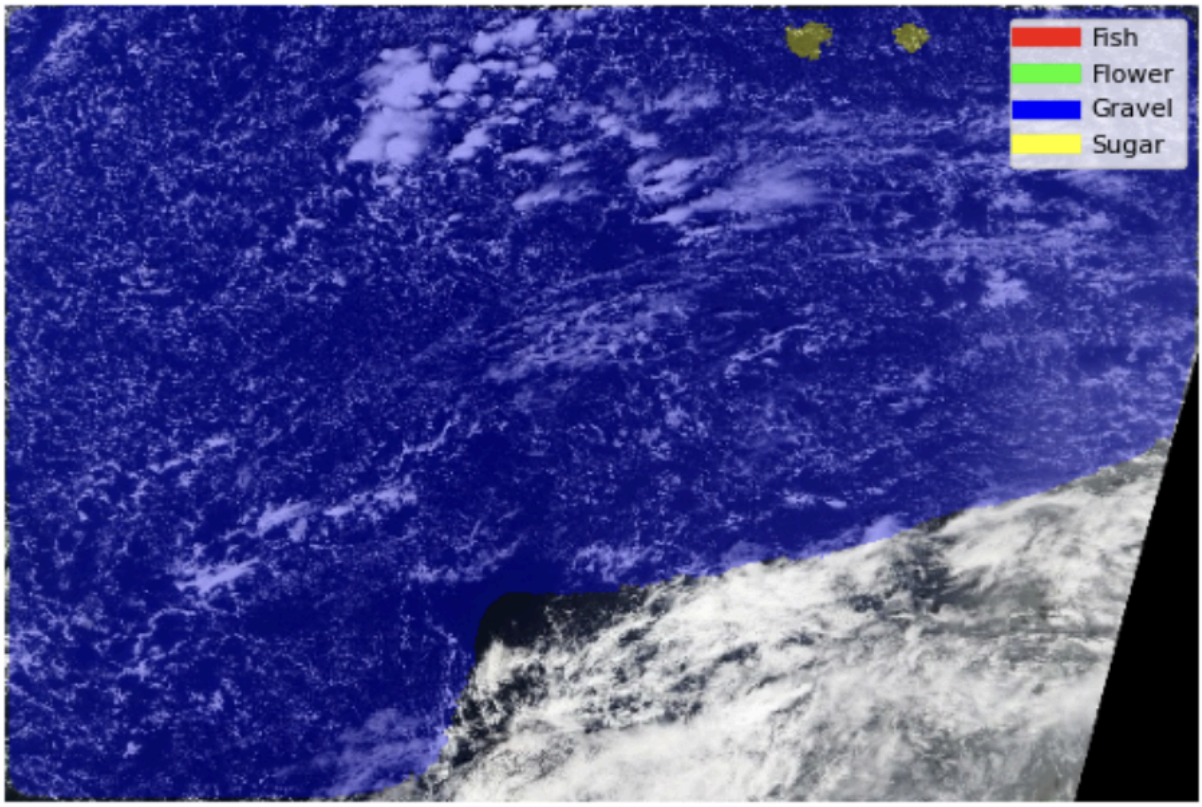
The test set contained 3,698 satellite images. The same preprocessing used for the validation data was applied to the test images, including resizing and ImageNet normalization.

The trained model generated four output channels corresponding to Fish, Flower, Gravel, and Sugar. A sigmoid function was applied to convert the model outputs into probabilities, and a threshold of 0.5 was used to obtain binary segmentation masks.

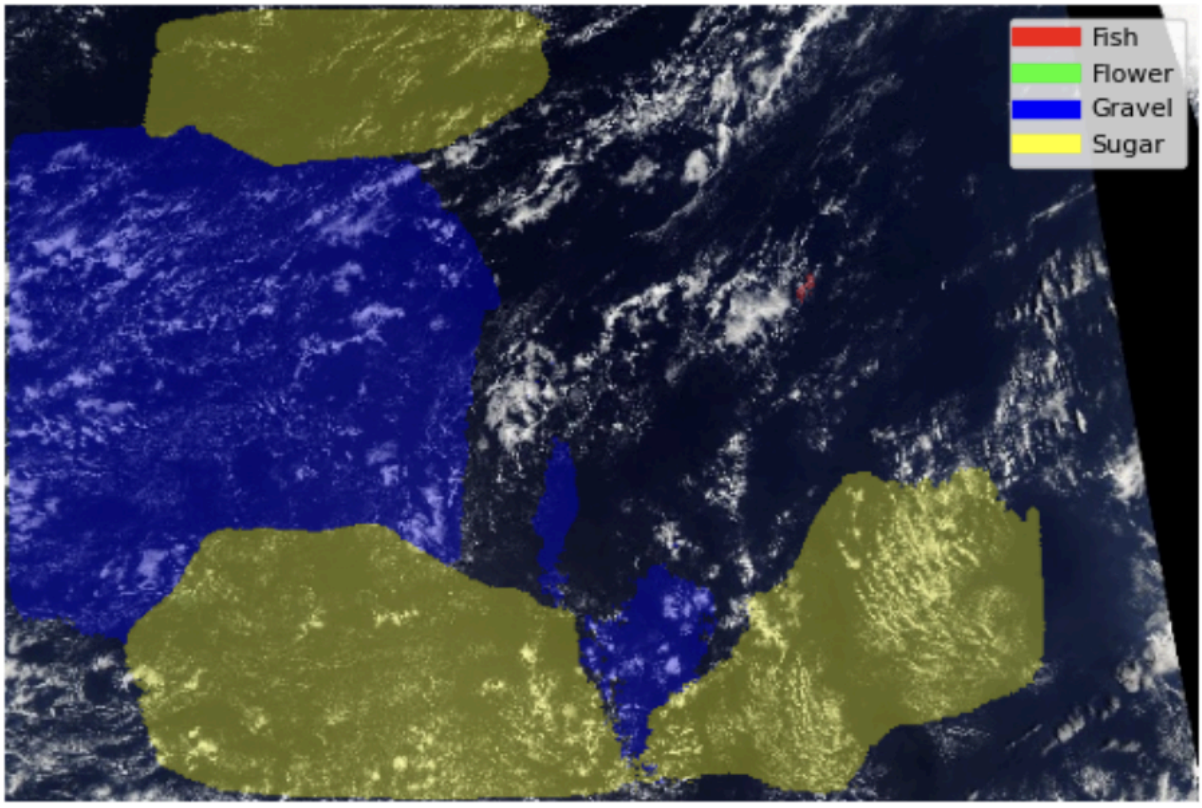
The predicted masks were then converted back to Run-Length Encoding (RLE) format. A submission file was created containing one prediction for each image and cloud category. The final submission contained 14,792 rows, corresponding to 3,698 test images × 4 cloud categories.

Figure 3.4 shows predictions generated by the selected U-Net with ResNet50 model on seven test images. For each image, the predicted regions of the four cloud categories are displayed on the original satellite image. The different colors represent the four cloud classes: Fish, Flower, Gravel, and Sugar. These examples provide a visual overview of the model's predictions on previously unseen test images.

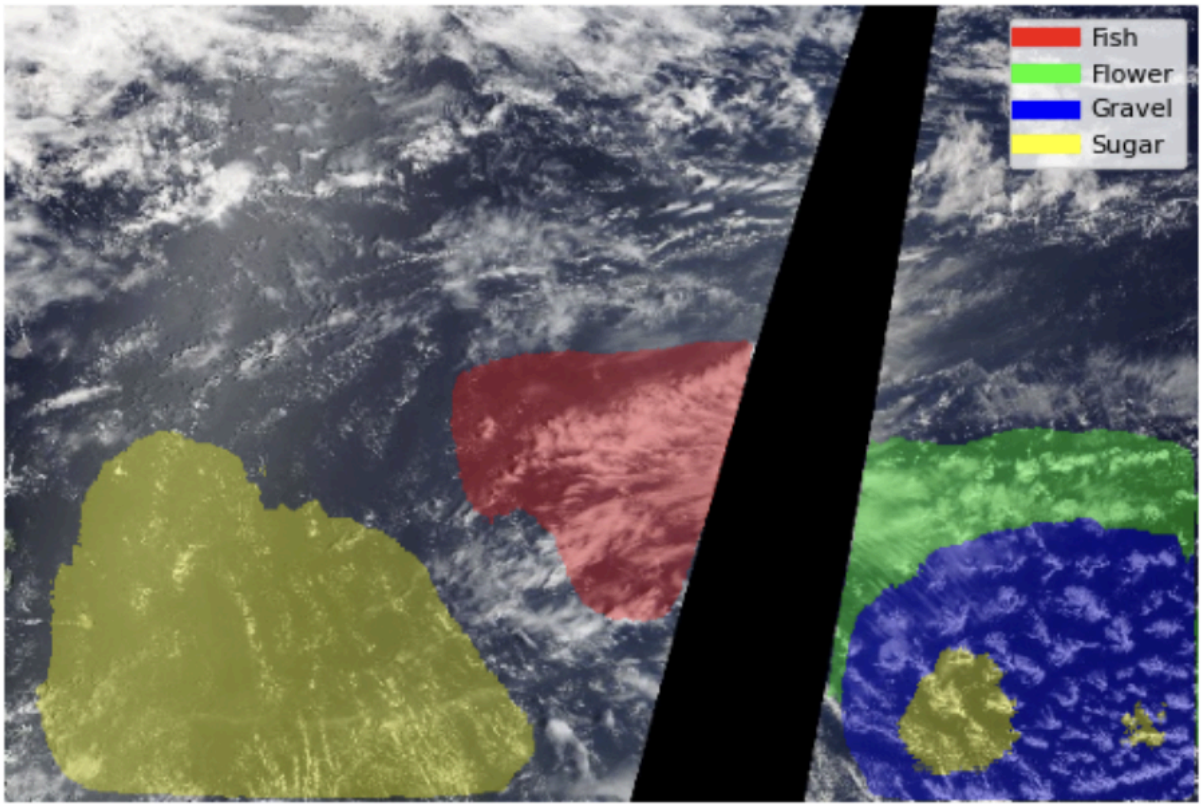
002f507.jpg | Detected: Gravel, Sugar



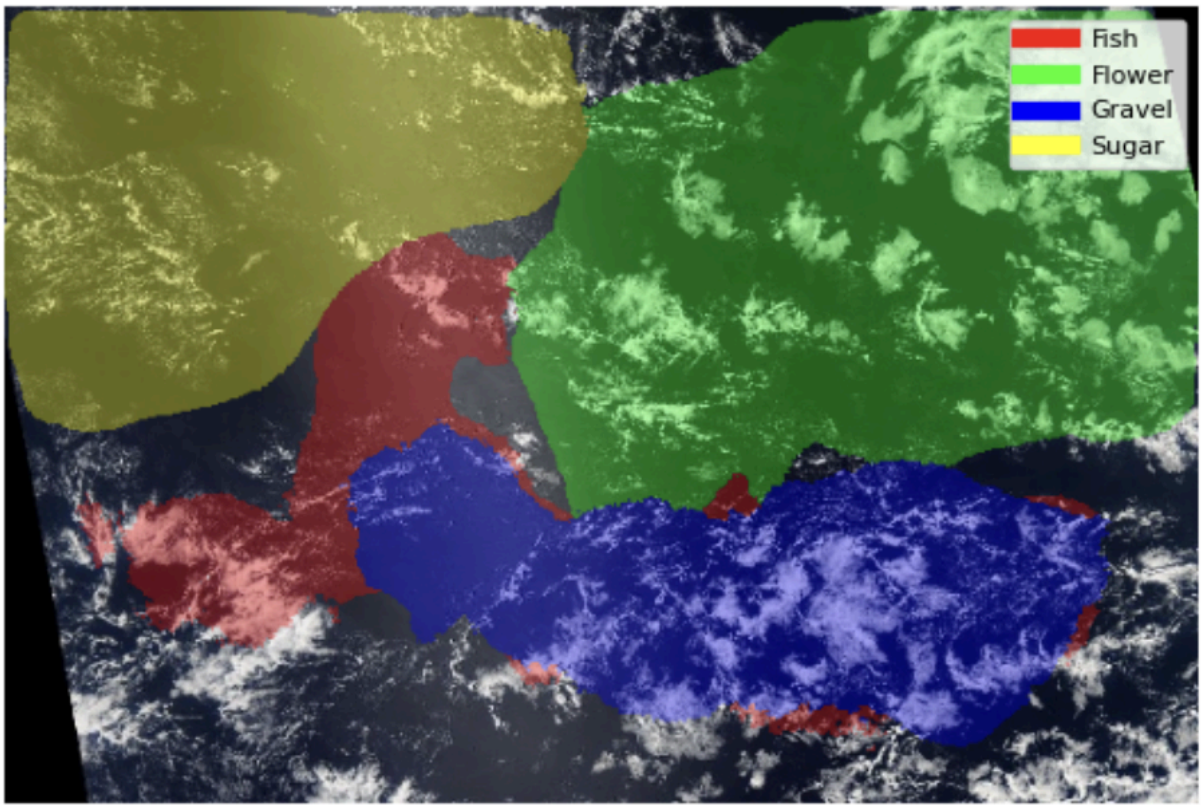
0035ae9.jpg | Detected: Fish, Gravel, Sugar



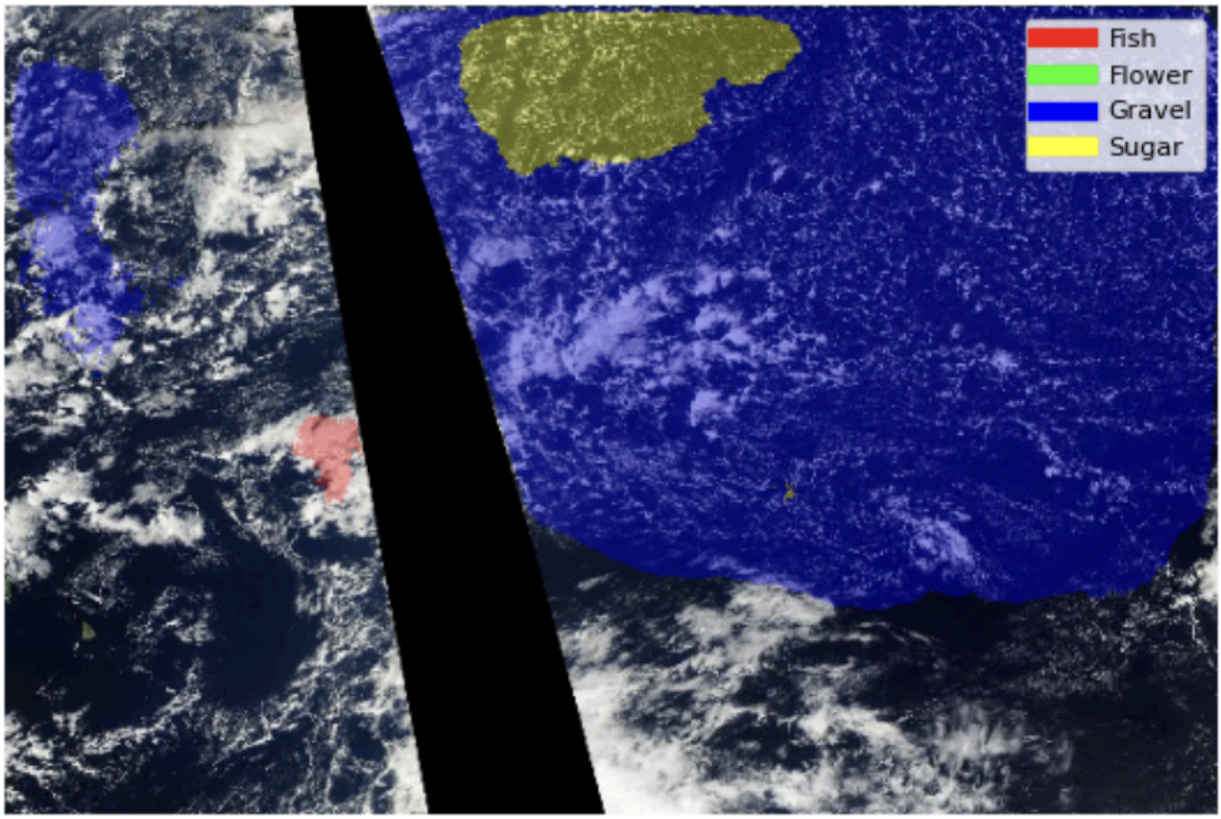
0038327.jpg | Detected: Fish, Flower, Gravel, Sugar



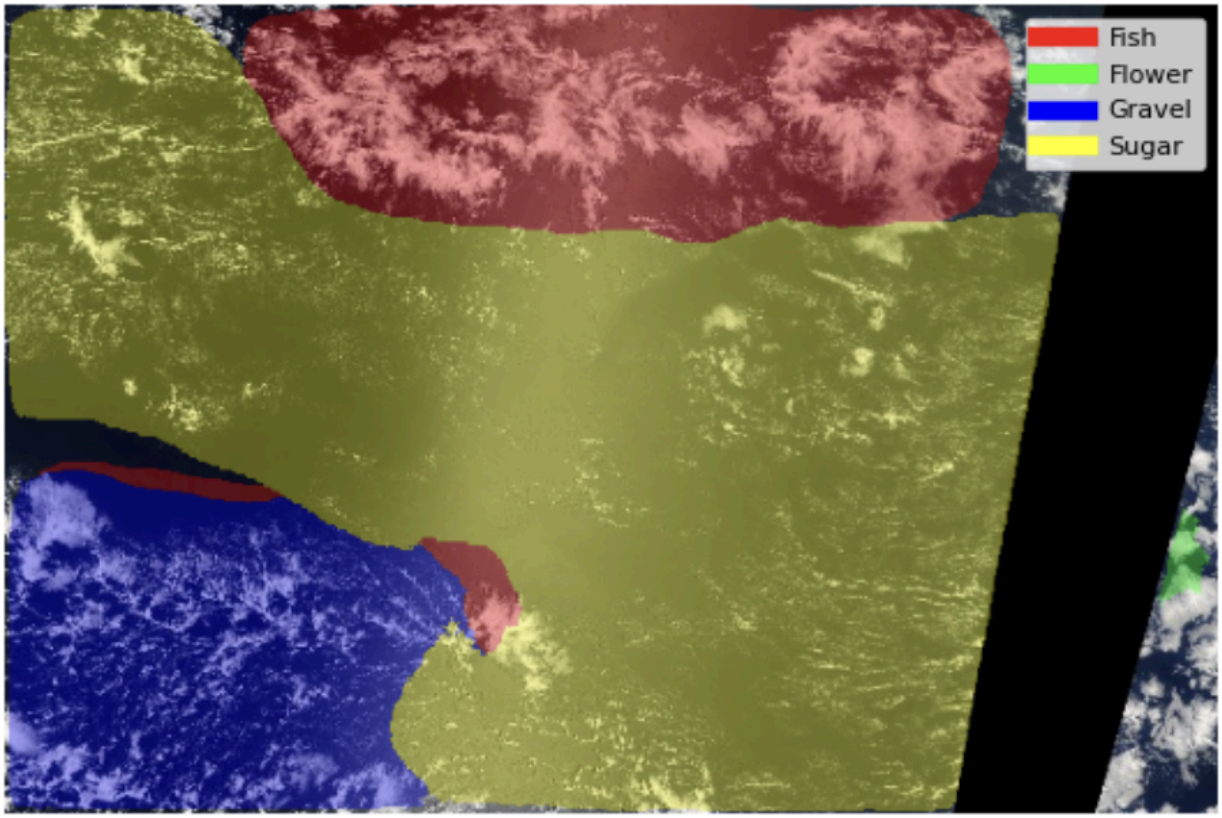
004f759.jpg | Detected: Fish, Flower, Gravel, Sugar



005ba08.jpg | Detected: Fish, Gravel, Sugar



006440a.jpg | Detected: Fish, Flower, Gravel, Sugar



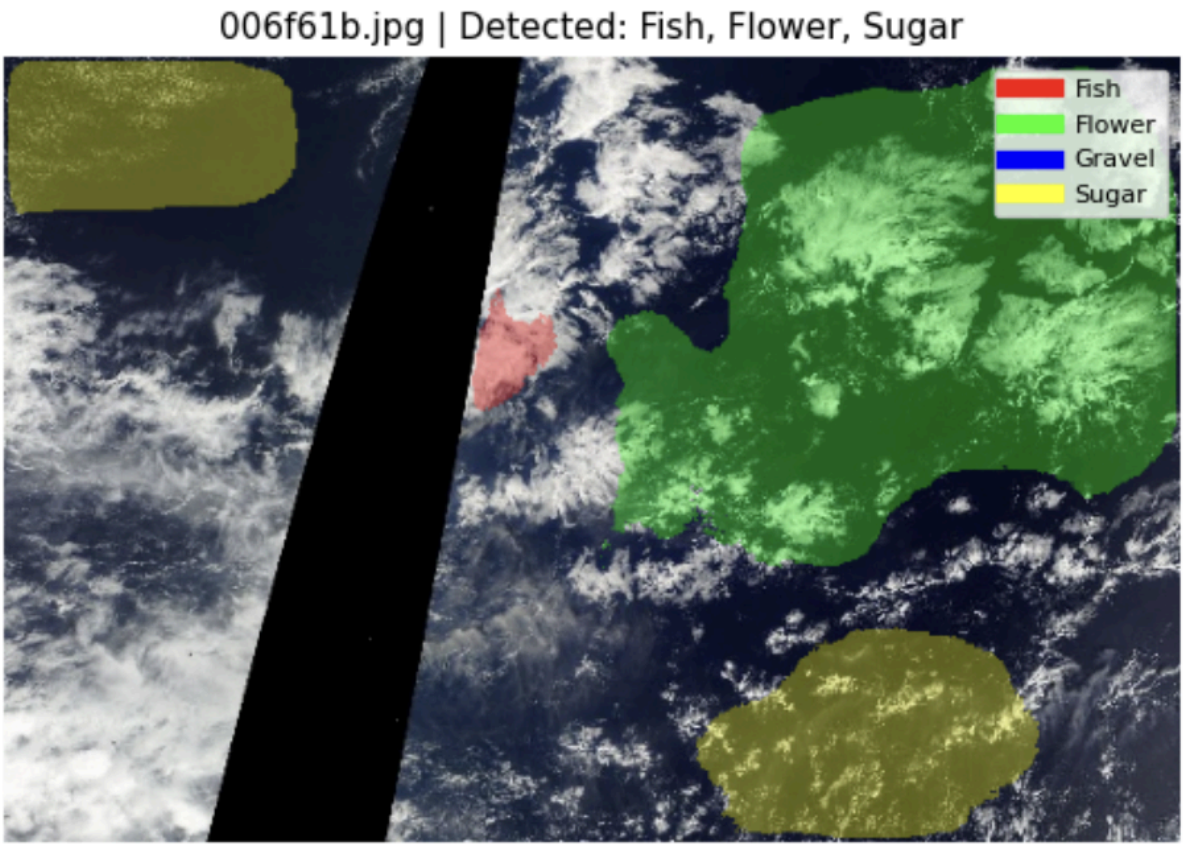


Figure 3.4. Example Test Set Predictions

3.9 Comparison of Model Predictions

The three segmentation models were compared using the same seven test images. The table shows the cloud categories detected by each model for each image.

Image	U-Net + ResNet50	U-Net + ResNet34	PSPNet + ResNet34
002f507.jpg	Gravel, Sugar	Gravel, Sugar	Gravel, Sugar
0035ae9.jpg	Fish, Gravel, Sugar	Fish, Gravel, Sugar	Gravel, Sugar
0038327.jpg	Fish, Flower, Gravel, Sugar	Flower, Gravel, Sugar	Fish, Flower, Sugar
004f759.jpg	Fish, Flower, Gravel, Sugar	Fish, Flower, Sugar	Fish, Flower, Sugar
005ba08.jpg	Fish, Gravel, Sugar	Fish, Gravel	Gravel
006440a.jpg	Fish, Flower, Gravel, Sugar	Fish, Flower, Gravel, Sugar	—
006f61b.jpg	Fish, Flower, Sugar	Fish, Flower, Gravel, Sugar	—

The results show that the models produced different predictions for several test images. **U-Net with ResNet50** generally detected more cloud categories than the other two models. However, this comparison only shows the detected categories and does not indicate which prediction is more accurate because ground-truth masks are not available for the test images.

3.10 Final Model Comparison

Three segmentation models were compared using the overall Macro Dice score obtained on the validation set.

Model	Encoder	Best Validation Dice
U-Net	ResNet34	0.5974
U-Net	**ResNet50**	**0.6118**
PSPNet	ResNet34	0.6114

U-Net with ResNet50 achieved the highest validation Dice score of 0.6118, followed very closely by PSPNet with ResNet34 with 0.6114. U-Net with ResNet34 achieved a score of 0.5974.

Based on the validation Dice score, U-Net with ResNet50 was selected as the best-performing model. However, the difference between **U-Net with ResNet50** and PSPNet with ResNet34 is very small, indicating that their validation performance was very similar.

3.11 Conclusion of Model Development

Three segmentation models were developed and evaluated in this project: U-Net with ResNet34, U-Net with ResNet50, and PSPNet with ResNet34.

Based on the validation results, **U-Net with ResNet50 achieved the highest Macro Dice score of 0.6118**. PSPNet with ResNet34 achieved a very similar score of 0.6114, while U-Net with ResNet34 achieved 0.5974.

Therefore, **U-Net with ResNet50 was selected as the final model** for generating predictions on the test images. The selected model was also used for the final test-set prediction and submission.

References

1. Kaggle. *Understanding Clouds from Satellite Images*. Kaggle Competition Dataset.
2. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI).
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
4. Iakubovskii, P. (2019). *Segmentation Models PyTorch*. GitHub.
5. Buslaev, A., et al. (2020). *Albumentations: Fast and Flexible Image Augmentations*. Information, 11(2), 125.
6. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems (NeurIPS).
7. Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. International Conference on Learning Representations (ICLR).